

HDMAS: Homogeneous Decentralized Multi-Agent System

Weikai Liao

Draft outline v0.1 — 2026-04-24

1 Problem

1.1 The wide-search workload

Consider a representative WideSearch query: *“List every Michelin-starred restaurant in Tokyo that closed in 2024, with name, ward, cuisine, star count, and closing date.”* Answering it requires touching 5–30 distinct sources — guide listings, restaurant homepages, news articles, social posts — pulling one row from each, and assembling the result into a single table. Two traits define this workload:

- **Broad horizontal coverage** — each row comes from a different source; missing one source loses one row.
- **Strict vertical verification** — scoring is at the cell level; one missing row or one wrong column is visible in the metric.

WideSearch packages 200 such queries (100 EN + 100 ZH) and reports row-level F1, item-level F1, and exact-match success rate (SR). It is the workload this paper targets.

1.2 Why a single agent struggles

The most natural way to attack a wide-search query is to give it to one agent and let it walk the sources sequentially. This is also the configuration almost every public leaderboard entry uses. It underperforms for three structural reasons, all rooted in the same fact: **a single agent must hold the entire run inside one context window.**

1. **Context-length pressure.** Each source contributes raw search hits, fetched pages, and intermediate notes. By source 15–20 the working context is dominated by earlier evidence, leaving little budget for the next query and for the final table assembly.
2. **Serial latency.** Wall-clock time scales linearly in the number of sources. On a 30-source query this turns into tens of minutes, well past the point where the agent’s own step budget runs out.
3. **Attention drift → early convergence.** As the trajectory grows, the model’s attention on the *original* subtask list weakens. Empirically, the agent declares itself done while 3–8 rows are still missing — the dominant single-agent failure mode in our logs. Adding more steps does not fix this: the agent has already convinced itself the table is complete.

These are not prompt-engineering bugs; they are properties of running many loosely coupled lookups inside one growing trajectory. The shape of the task — many independent lookups, then one tabular merge — is exactly what parallelism is good at, and a single agent cannot exploit it.

1.3 Multi-agent fixes that, but the *conventional* shape brings its own ceiling

Going to N agents in parallel solves §1.2 directly: each peer’s context only carries its own slice of sources, latency drops $\sim N\times$, and “did we cover everything?” becomes a question about a shared artifact instead of one fading attention span. Multi-agent is clearly the right shape for wide search; the design question is **how the N agents are organized**.

The default, almost-always-chosen answer is a **centralized MAS**: a privileged coordinator (planner / router / critic) decomposes the query up front, dispatches subtasks to workers, merges their results, and decides when to stop. AutoGen GroupChat, Magentic-One, CrewAI, and MetaGPT are this shape; N is typically held under 5 because that is what one planner can manage in one context. This pattern works, but in our wide-search setting it has three structural problems that all trace back to the same thing — *one privileged agent decides for everyone*:

- **The planner is a single point of failure.** Every worker’s behaviour is downstream of one LLM call. If the planner forgets a source, splits the query along the wrong axis, or calls the run done too early, the mistake is amplified across all N workers and shows up as N missed cells, not one. There is no peer with the authority — or the context — to override that decision, because correcting it would mean re-running the planner. On a per-cell metric the asymmetry is brutal: one bad coordinator turn = many immediately visible failures.
- **The coordination layer demands more from the planner than the workers.** A useful planner has to simultaneously hold the full enumeration of sources, the current state of every worker, the partial results coming back, and the open questions still to dispatch — exactly the context-pressure pathology we just escaped in §1.2, now relocated onto a single agent that *also* has to reason about scheduling. Workers can be small focused contexts; the planner can’t. So the per-call cost and per-call failure rate of the most critical agent in the system are the highest in the system.
- **Wide-search work is not knowable up front, and a planner-shaped MAS pays for every adjustment.** Empirically, on WideSearch we see a recurring pattern: an agent will hammer one attribute of one entity for many iterations and not find it (an obscure fee schedule, a missing field on a vendor page), while a *different* agent trying a different angle (a different search query, a different source, a different language) finds it on its first try. Wide search has a long tail of items where retrieval succeeds *only* under retry-with-diversity. Centralized MAS *can* handle this — the planner just re-plans — but every re-plan is a global synchronization point: workers wait, the planner re-reads the entire growing state, and the system spends an LLM call to do what is structurally a “let another peer try a different angle” decision. The protocol cost of one re-plan is small; the cost compounds badly when re-plans are frequent, which on WideSearch they are.

So the real question isn’t single-vs-multi; it’s **centralized-vs-decentralized** within the multi-agent design space — and specifically: *do you want one agent’s worst LLM call to bind everyone, or do you want a protocol where any peer’s good call can rescue any other peer’s bad one?*

1.4 Our approach: a flat, blackboard-coordinated team

HDMA takes the decentralized side: instead of a coordinator, all coordination state lives in a single shared file (`blackboard.json`) guarded by an OS-level file lock, and N **identical** peers — same prompt, same toolset, no roles — each run the same loop end-to-end. Every peer, at the top of every iteration, locks the blackboard, re-derives the next claim from the original query and the current

`task_log`, writes back exactly one new `in_progress` entry, releases the lock, and goes off to execute. Planning is not eliminated; it is **distributed across peers and scoped to the next claim**, so no single context ever holds the whole plan. Dispatch — “which agent runs this” — collapses into a `flock()`. The coordinator is not absent; it is replaced by **the lock + the blackboard’s invariants**.

The honest costs of this design are **lock contention** on shared writable state and the fact that **distributed termination is harder than central termination** (we need an explicit `vote_stop` + `wake_agent` protocol, §2.2). The bet of this paper is that on wide-search workloads the gains outweigh those costs. §2 lays out exactly what machinery this requires (it is small); §3 reports the numbers; §4.7 returns to *when* this trade-off favors decentralization and when it doesn’t.

2 Method — HDMAS

HDMAS is built on a **flat, fully symmetric multi-agent architecture**. Instead of a lead orchestrator that decomposes the query, dispatches subtasks, and tracks progress on behalf of the team, all coordination state is externalized to a single shared file — the **blackboard** — guarded by an OS-level file lock. There is no planner agent, no router, no critic, and no message bus. Every agent runs the same system prompt and the same toolset, and the question “what should be done next, and by whom?” is answered not by an LLM call but by a peer reading the blackboard under the lock and appending its own claim.

The work of each agent is structured as a single repeating loop with two LLM-driven phases (illustrated in the figure above): an **outer claim phase**, in which the agent locks the blackboard, re-derives the full subtask list from the current query, diffs it against what is already logged, and writes back exactly one new `in_progress` entry under its own id; and an **inner execute phase**, in which the agent runs the claimed subtask using its own tools, stages intermediate artifacts under its private workspace, and finally locks the blackboard again to flip its entry to `done`. When an agent believes the run is complete it enters a third, lightweight phase: it rescans the blackboard, verifies that the staged answer is consistent with `task_log`, and calls `vote_stop`. The run terminates when every agent has voted to stop.

2.1 Components

HDMAS consists of the following components. Unlike Magentic-One’s role-specialized agents, only the first is an LLM-driven actor; the rest are coordination substrate.

- **Peer Agent.** An LLM-driven agent that runs the HDMAS main loop end-to-end on its own. Each peer enumerates subtasks from the query, claims one by appending to the blackboard, executes it using the built-in Copilot tools (web search, file I/O, shell, Python), writes intermediate evidence into its private workspace, and marks the entry `done`. All N peers share an identical system prompt and identical tool list — there are no roles such as “researcher”, “writer”, or “critic”. A peer may also call `vote_stop` to declare the run complete, or `wake_agent` to re-activate a sleeping peer when it discovers new work.
- **Blackboard (`blackboard.json`).** The single source of truth for coordination. It contains the original query, an append-only `task_log` array of `{task, agent, status, outcome, summary}` entries, and a per-agent status map. The `task_log` plays the role that Magentic-One’s Task Ledger and Progress Ledger play together — it is simultaneously the plan and the progress record — but it is owned by no one, continuously re-derived by every peer at the top of every iteration,

HDMAS — Symmetric Multi-Agent Architecture with Shared Blackboard

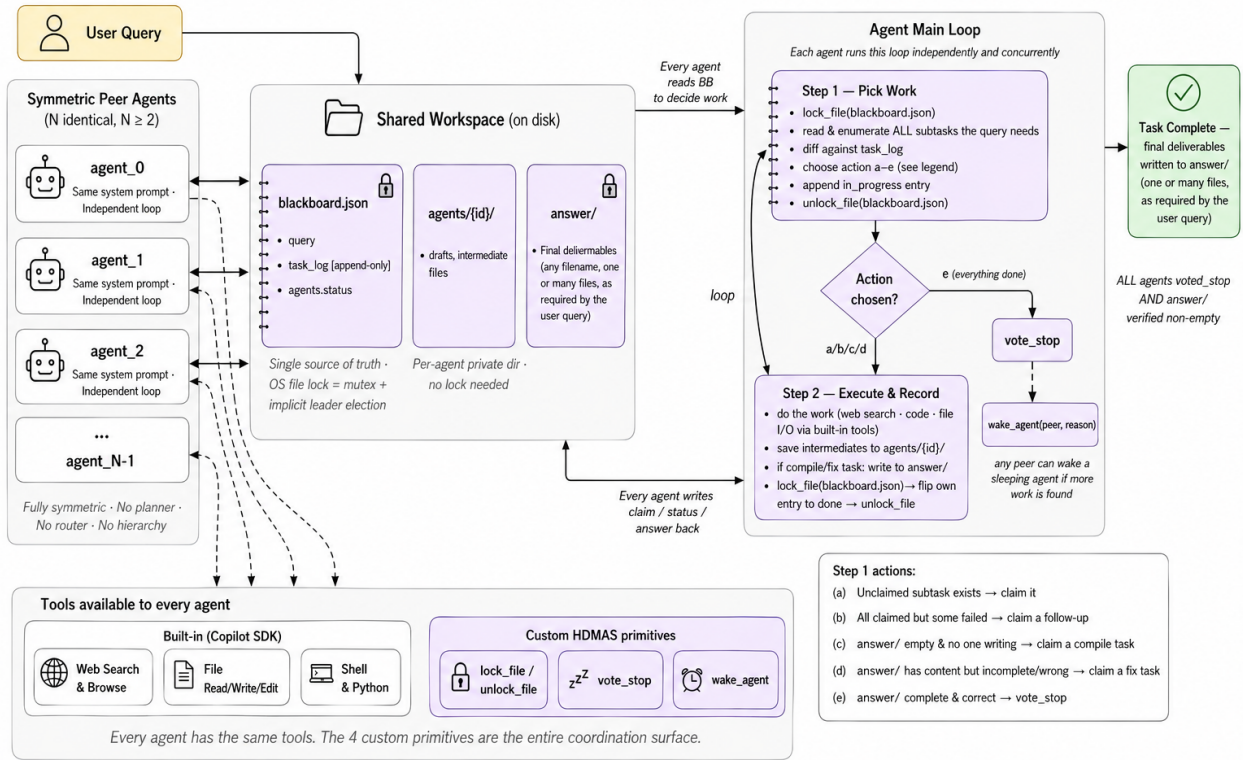


Figure 1: HDMAS architecture

and write-restricted so that each agent may only modify entries whose **agent** field equals its own id.

- **File Lock.** An OS-level advisory lock on `blackboard.json`. Every read-modify-write of the blackboard must happen inside the lock, and the lock is the only mechanism that prevents two peers from claiming the same subtask. It also provides implicit leader election: at any instant, whichever agent is holding the lock defines what `task_log` currently *is*, and all other peers see a consistent snapshot the moment they acquire it next.
- **Per-agent workspace (`agents/{id}/`).** A private directory in which each peer stages search hits, fetched pages, and intermediate notes for its own subtasks. Workspaces are write-isolated by id, so peers never overwrite each other’s evidence and contention is confined to the blackboard and the final answer area.
- **Answer staging area (`answer/`).** A shared directory where peers assemble the final tabular output. This is the second contended region (after the blackboard) and the source of the “multi-writer overwrite-and-drift” failure mode discussed in §3.4.

Together, these components give every peer the full set of capabilities needed to make end-to-end progress on a wide-search query — discover sources, fetch and parse them, write rows, and decide when to stop — without ever consulting a central coordinator.

2.2 Coordination primitives

Beyond the tools the Copilot backend already provides (web search, file I/O, shell, Python, code execution), HDMAS adds only four custom primitives:

- `lock_file(path)` / `unlock_file(path)` — acquire and release an OS-level advisory lock on a file. Used almost exclusively on `blackboard.json`, occasionally on individual files in `answer/`.
- `vote_stop(reason)` — the calling agent puts itself to sleep after declaring the run complete. The run as a whole terminates only when every peer is in this state.
- `wake_agent(id, reason)` — wake a sleeping peer. Used when a still-active peer discovers, after others have voted to stop, that the answer is in fact incomplete.

We deliberately do not introduce a planner agent, a message bus, a typed task DAG, a vector store, or any fine-tuning pipeline. The blackboard plus the lock already expresses both “what is being done” and “who is doing it”; LLM-enumerated subtasks subsume the role of a hand-coded DAG; WideSearch is an open-web benchmark and the built-in search is sufficient; and we want the result to be attributable to the architecture, not to model-level customization. The Copilot backend model is used as-is for every peer.

2.3 The agent loop and lock protocol

Each peer runs the following loop until it votes to stop:

```
loop:
  Step 1 -- claim (lock held throughout):
    lock_file(blackboard.json)
    read blackboard.json
    enumerate the full subtask list from ‘query’
```

```

diff against task_log
pick exactly one action: claim a missing subtask,
                        redo a stalled one,
                        extend the answer, etc.
append one in_progress entry under own id
write blackboard.json
unlock_file(blackboard.json)

Step 2 -- execute (lock NOT held):
run the claimed subtask using built-in tools
stage intermediates under agents/{id}/, write rows into answer/
lock_file(blackboard.json)
    flip own entry to done, attach outcome + summary
unlock_file(blackboard.json)

Step 3 -- stop check (only when no work remains in Step 1):
rescan blackboard, verify answer/ matches task_log
vote_stop(reason)

```

The lock is held **only across reads and writes of the blackboard, never across LLM calls or tool execution**. This matters in two directions. On one side, it must be held *across* the read and the subsequent write in Step 1: the agent has to see the latest `task_log` and append its claim atomically, otherwise two peers reading the same snapshot will claim the same subtask (the split-brain failure mode discussed in §4.5). This is why the read of the blackboard is effectively **serialized across peers** — at any instant only one agent is reading-and-deciding, the others are blocked on `lock_file`. On the other side, the lock is explicitly released *before* the agent does any real work in Step 2: web fetches, page parsing, and writing into `agents/{id}/` and `answer/` all run with the blackboard unlocked, so peers execute in parallel and only re-serialize for the short flip-to-done write at the end.

The lock thus plays a dual role: it is a **mutual-exclusion primitive** that keeps `task_log` consistent, and it is an **implicit leader-election primitive** — whichever agent grabs the lock first defines what the team’s plan currently is, and the next agent in line sees that decision before making its own. No agent is permanently “the leader”; leadership rotates on every claim.

3 Results

3.1 WideSearch (en+zh, 200 items)

Metric: Avg@1. Leaderboard numbers are Avg@4 — order-of-magnitude reference only.

Headline (n=200).

System	SR	Row-F1	Item-F1
HDMAS_COPILOT (3 agents)	0.125	0.556	0.733
BARE_COPILOT (single agent)	0.115	0.499	0.679
HDMAS_COPILOT (8 agents)	0.090	0.591	0.781

Fair comparison — EN subset (n=100).

System	SR	Row-F1	Item-F1
HDMAS_COPILOT (3 agents)	0.140	0.532	0.753
BARE_COPILOT	0.140	0.509	0.741
HDMAS_COPILOT (8 agents)	0.110	0.523	0.766

Fair comparison — ZH subset (n=100).

System	SR	Row-F1	Item-F1
HDMAS_COPILOT (3 agents)	0.110	0.581	0.714
BARE_COPILOT	0.090	0.490	0.617
HDMAS_COPILOT (8 agents)	0.070	0.658	0.797

Leaderboard reference (leaderboard = Avg@4).

System	SR	Row-F1	Item-F1
HDMAS_COPILOT 3a (Avg@1, ours)	12.5	55.6	73.3
Best Multi-Agent on board (OpenAI o3 MA, Avg@4)	5.1	37.8	57.3
Best Single-Agent on board (OpenAI o3 SA, Avg@4)	4.5	34.0	52.6

$\triangle$ Avg@1 vs Avg@4 are not strictly comparable; the underlying model versions may differ by a generation. Listed under verification TODO.

3.2 BARE_COPILOT — single-agent baseline is already strong

BARE alone beats every entry on the SA leaderboard by 2–3 $\times$. The Copilot backend + built-in tools have a high ceiling on their own; HDMAS’s gain has to be measured *on top of* that.

3.3 HDMAS_COPILOT (3 agents) — the gain comes from ZH

- EN vs BARE: SR flat (+0.0), Row-F1 +2.3 pt
- ZH vs BARE: SR +2.0 pt, Row-F1 +9.1 pt, Item-F1 +9.7 pt
- Multi-agent coordination cost is recovered only on Chinese queries.

3.4 HDMAS_COPILOT (8 agents) — the “fine but incomplete” anti-pattern

EN fair comparison:

Metric	3a	8a	Δ
Item-F1	0.753	0.766	+1.3 pt
Row-F1	0.532	0.523	−0.9 pt
SR	0.140	0.110	−3.0 pt

ZH fair comparison:

Metric	3a	8a	Δ
Item-F1	0.714	0.797	+8.3 pt
Row-F1	0.581	0.658	+7.7 pt
SR	0.110	0.070	-4.0 pt

The pattern is consistent across both subsets: **cell-level metrics keep climbing while row-level SR collapses** — 8a finds *more* of the right cells than 3a, but the final answer file ends up missing or corrupting one or two rows so the all-or-nothing SR drops. The cause is contention at the **answer-write** stage, not at retrieval. Log statistics (EN run):

- 8a has **82.4% of cases** in which **answer/*** is locked by ≥ 2 agents.
- Worst case: 7 agents fight over a single **final.md**.
- Multi-writer subset SR is **7.8%**, single-writer subset is **19.0%**.

Conclusion: 3a $\rightarrow$ 8a is not worth it; stop at 3a.

4 Insights

Metric primer. WideSearch reports three numbers per case, then averages over the suite.

- **SR** (success rate): 1 if the produced table matches the gold table *exactly* after normalization, else 0. **Strict, all-or-nothing, row-and-cell.**
- **Row-F1**: precision/recall/F1 over rows that match end-to-end. One wrong cell kills the row.
- **Item-F1**: precision/recall/F1 over individual cells. A row that is mostly right still contributes its correct cells.

The three metrics behave differently under contention: a system can simultaneously have **rising Item-F1, rising Row-F1, and falling SR**. Keep this in mind for the insights below.

The insights below are stated as conjectures about wide, parallel, fan-out workloads in general; the WideSearch numbers in §3 are evidence, not the scope of the claim.

4.1 Strict and averaged metrics diverge under coordination contention

Once multiple peers share any writable artifact (a row file, a JSON record, a table cell), the system has two distinct failure surfaces:

- **Coverage failures** (a cell never gets written) hurt every metric proportionally — an averaged metric like Item-F1 absorbs them gracefully.
- **Commit failures** (a cell gets written, then partially overwritten or reordered by a racing peer) are nearly invisible to averaged metrics — most cells survive — but **fatal to strict-match metrics**, because one corrupted row turns the whole answer into a 0.

So as you add peers, coverage failures fall faster than commit failures rise; averaged metrics climb while the strict-match metric can drop. **Reporting only one of the two systematically misleads.** Wide-fanout systems should report both an averaged metric (to credit partial work) and a strict metric (to expose commit pathologies); choosing N by averaged-metric-only will hide a real production regression.

4.2 Adding peers trades retrieval headroom for assembly contention

The benefit of an extra peer is bounded by how much *new* work the workload still has — once retrieval is saturated, additional peers contribute almost nothing on the coverage side but still pay the full cost on the contention side. The bottleneck therefore migrates predictably with N :

N	Dominant bottleneck	What another peer buys you
1	retrieval / coverage	a lot
few	light commit contention	some on coverage, a little overhead
many	heavy commit contention on output	almost nothing on coverage, much more contention

The right N is therefore **the smallest N at which the workload’s natural retrieval parallelism is roughly saturated** — beyond it, every added peer is net-negative on strict-match quality and net-positive on cost. Picking N is a per-workload calibration, not a knob to maximize. Symptoms that you have already passed the right N : strict-match metric drops, tail latency balloons, multi-writer events become the modal failure mode in logs.

4.3 Cost grows faster than peer count, and the gap measures wasted work

Wall-time scales roughly linearly with N (parallelism is partial — the lock that protects the shared coordination state serializes the read-decide-claim window), but **tool calls / tokens / dollars scale super-linearly**. On WideSearch, $3a \rightarrow 8a$ is $\approx 2\times$ wall-time and $\approx 2.7\times$ tool calls (avg 316 $\rightarrow$ 851 per case; total ≈ 59.6 h $\rightarrow \approx 117.5$ h over $n = 200$).

The super-linear gap is itself a useful diagnostic: under contention, peers re-read the shared state, re-claim after preempted writes, and re-issue similar work. The faster cost grows above N , the more redundant work the architecture is generating. A coordination layer that stays at exactly $\times N$ tool calls is honest scaling; one that goes to $\times 1.5N$ or $\times 2N$ is paying compound interest on contention. The Pareto-improving direction is therefore not “add more peers” but **reduce the contention surface** (finer-grained locks, single-writer commit phases, an explicit verifier pass) — the same fixes that would raise the strict-match metric also flatten the cost curve.

4.4 What HDMAS is actually for: failure containment, not raw speed

A fair pushback after seeing §3 and §4.1–4.3: if cell-level wins are bought at $\approx 2\times$ cost, and a planner-worker MAS can also re-plan when things go wrong, what’s the unique value of a flat blackboard?

The honest answer is that **HDMAS is not a faster MAS — it’s a MAS that degrades gracefully under conditions where planner-worker fails catastrophically**. The benefit comes from one structural fact: in a centralized MAS, every coordinator decision binds N workers, so one bad decision is amplified $N\times$; in a flat blackboard team, every decision is local to its taker and correctable by the next read of the shared state.

Concretely, three claims survive scrutiny:

1. **Failures don’t get amplified.** A planner that mis-decomposes (forgets a source, splits the wrong row, calls the run done too early) propagates that mistake to every worker. In HDMAS the same mistake — a peer missing a source, claiming the wrong cell, voting **stop** prematurely — only consumes one peer’s iteration; the next peer that locks the shared state re-derives the work list and corrects it. This is a *correctness* property, not a performance one, and it doesn’t degrade with scale.

2. **Late-arriving evidence is absorbed without a barrier.** A peer that discovers source k' mid-execution just appends a claim on its next iteration — no replanning round-trip, because there is no plan to replan. Yes, planner-worker can re-plan, but every re-plan is a global synchronization point: workers wait while the planner re-decomposes, and the planner has to hold the entire current state (query + all subtasks + all results so far) in one context, which after a few re-plans reproduces the single-agent context-pressure pathology you went multi-agent to escape. HDMAS pays one lock-and-append on the blackboard.
3. **Coordination invariants become protocol, not LLM judgement.** When the planner re-plans, “is task T still in-flight?”, “did this ‘no results’ mean empty or unreachable?”, “the new evidence contradicts T_2 ’s earlier finding — cancel, retry, or trust the new one?” are all distributed-consensus questions answered by whatever the planner LLM decides on each call — non-deterministic and non-auditable. In HDMAS the same questions are answered by the protocol: lease expiry on claims, explicit `outcome={empty|error}` written before lock release, and contradictions resolved by an explicit append in `task_log` that any later peer can audit.

What HDMAS is *not* doing: this is **not ensemble diversity** — all peers share one model and have correlated biases, so we don’t get variance-reduction from “many independent voters”. The benefit is purely **containment**: keeping any one peer’s failure from binding the others.

Where HDMAS does *not* win. Centralized is better when the cost of one coordinator decision is small relative to the cost of duplicated or wasted worker work — strong task-graph dependencies (workers must serialize anyway), small N , termination naturally signalled by the artifact structure, or workers that genuinely need different roles. Software engineering with explicit specs, role-played simulations, and tightly sequential workflows are all centralized-friendly.

The clean way to state the regime where HDMAS pays off:

- the work is wide and weakly coupled,
- the failure metric is per-item coverage,
- individual peer failures are common (timeouts, hallucinated stops, partial answers),
- the task graph isn’t fully knowable up front (sources discovered mid-run).

WideSearch lands in that regime, which is why §3 came out the way it did. GAIA and other benchmarks where any single missed step zeroes the run will probably show the same shape; production agents that have to be robust to flaky tools and unpredictable web sources are the real target.